HelixVPN - Deep research

Here's your complete research-based guide to setting up a dedicated-host VPN service that provides full network access without touching the main router, while offering Mullvad-like features (including QUIC protocol, no-logging, kill switch, and more).

Core Principle

Since you cannot configure port forwarding on the main router, the solution is to reverse the connection: Your internal dedicated host actively establishes an outbound tunnel to a public VPS (Virtual Private Server). The VPS acts as a relay/gateway. All remote clients then connect to the VPS, which forwards traffic to/from your internal network.

This completely bypasses the router's inbound firewall restrictions.

Two Main Architectural Models

Model Description Tools Hub-and-Spoke All peers (internal host, remote clients) connect to a central VPS. Traffic flows through the VPS. WireGuard, OpenVPN Peer-to-Peer (Mesh) Peers establish direct connections after NAT traversal (hole punching). The VPS only coordinates. nospoon, Tailscale, ZeroTier

For maximum similarity to Mullvad, the hub-and-spoke model with a dedicated VPS is the most straightforward, because Mullvad itself uses a central gateway model.

Recommended Technology Stack

Feature WireGuard OpenVPN Hysteria nospoon QUIC support ❌ (UDP, but not QUIC) ❌ ✅ (built on QUIC) ❌ Traffic obfuscation ❌ (plain UDP) Optional (via tls-crypt) ✅ (masquerades as HTTP/3) ❌ Performance Excellent Good Excellent (especially on lossy networks) Good Ease of setup Very easy Moderate Moderate Very easy Mullvad-like features Yes (kill-switch, split-tunnel) Yes Yes (plus anti-censorship) Limited

My strong recommendation: Use Hysteria if you want QUIC + obfuscation + performance. Use WireGuard if simplicity and wide client support are your priority (you can add obfuscation with tools like AmneziaWG).

Step-by-Step Deployment (with WireGuard as the baseline)

1. Acquire a VPS · Choose a provider with a public IP (e.g., DigitalOcean, Vultr, AWS EC2). · Minimum specs: 1 vCPU, 1 GB RAM, ~$5-6/month.
2. Install WireGuard on both the VPS and your internal dedicated host (follow OS-specific instructions).
3. Configure the VPS as the server · Generate server private/public keys. · Edit /etc/wireguard/wg0.conf:

```
[Interface]
Address = 10.0.0.1/24
ListenPort = 51820
PrivateKey = <server_private_key>
```

```
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A
POSTROUTING -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat
-D POSTROUTING -o eth0 -j MASQUERADE

[Peer]
PublicKey = <client_public_key>
AllowedIPs = 10.0.0.2/32
```

· Enable IP forwarding (net.ipv4.ip_forward=1) and configure firewall to allow UDP port 51820.

4. Configure the internal host as a client · Generate its own keys. · /etc/wireguard/wg0.conf:

```
[Interface]
Address = 10.0.0.2/24
PrivateKey = <client_private_key>
[Peer]
PublicKey = <server_public_key>
Endpoint = <VPS_PUBLIC_IP>:51820
AllowedIPs = 0.0.0.0/0   # full tunnel - all traffic goes via VPN
PersistentKeepalive = 25
```

5. Start the tunnel on both ends. The internal host will connect outbound to the VPS, so no router port-forwarding is needed.
6. Route internal network access · If you want to reach other devices inside your home network (e.g., 192.168.1.x), add static routes on the VPS or use NAT. For a full network, you can set up the internal host as a gateway (enable IP forwarding and masquerade for the LAN).

Adding QUIC Protocol (to be like Mullvad)

Mullvad supports QUIC in their app (though not all platforms). To emulate this:

· Use Hysteria instead of WireGuard. Hysteria is a QUIC-based proxy/VPN that wraps traffic in HTTP/3, making it look like normal web traffic. · Setup overview: · Install Hysteria on VPS (server) and on your internal host (client). · Configure server with an acme certificate (or self-signed) to enable TLS. · The server listens on UDP/443 (or any port). Clients connect with the same. · Hysteria also provides bandwidth-aware congestion control, great for high-latency networks. · Alternative: Use Quicly (QUIC library) or WireGuard over QUIC (experimental), but Hysteria is production-ready.

Mullvad-like Features & How to Implement

Feature Implementation No-logging policy Self-hosted – you control logging; set log levels to “none”. Kill-switch Use wg-quick or firewall rules that block non-VPN traffic if the tunnel drops. Split-tunneling Adjust AllowedIPs to route only specific subnets via VPN. Multi-hop Chain two VPS instances (not trivial, but doable with WireGuard). Obfuscation Hysteria’s HTTP/3 masking, or use obfs4 with OpenVPN. Client GUI Use wg-easy for WireGuard or Hysteria’s official client apps.

Security & Maintenance

· VPS hardening: Disable root login, use SSH keys, install fail2ban, keep OS updated. · Key rotation: Regularly regenerate WireGuard keys or Hysteria certificates. · Monitoring: Use wg show or Hysteria's built-in status endpoint. · Performance tuning: Adjust MTU (e.g., 1420 for WireGuard over IPv4) and UDP buffer sizes.

Which One Should You Choose?

· If QUIC and obfuscation are non-negotiable → Hysteria. · If simplicity, wide client support, and proven stability are more important → WireGuard (with optional AmneziaWG for obfuscation). · If you want zero configuration and P2P direct connections → Tailscale (but it's a managed service, not fully self-hosted).

Final Verdict

Your use-case is perfectly feasible without touching the main router. The reverse-tunnel approach with a VPS is the industry-standard solution. For a "Mullvad-like" experience with QUIC, Hysteria is the most direct path. If you prefer a simpler deployment, WireGuard still gives you 90% of the benefit with much easier management.

Would you like a detailed configuration file for Hysteria or a troubleshooting guide for the reverse-tunnel setup? I'm happy to dive deeper.

Here is the maximally detailed, step-by-step implementation guide for both Hysteria and WireGuard solutions, designed to give you full network access without touching your main router.

---

1. Core Architecture & Design Principles

· Reverse Tunnel: Your internal host initiates an outbound connection to a public VPS. This bypasses the need for inbound port forwarding on your home router. · Hub-and-Spoke: All traffic flows through the VPS (the "hub") to which your internal host and all remote clients connect. · VPS Requirements: A Linux VPS (Ubuntu 22.04 LTS recommended) with a public IP, root access, and open ports (UDP 443 for Hysteria, UDP 51820 for WireGuard).

---

2. Option A: Hysteria2 (Recommended for QUIC & Obfuscation)

Hysteria2 is built on QUIC, offers excellent performance on lossy networks, and includes built-in traffic obfuscation (Salamander).

2.1 VPS Server Setup (Automated)

The fastest way is to use a community script (ensure you review any script before running):

```
# Download and run the installer
bash <(curl -fsSL https://raw.githubusercontent.com/MeccCZ/hysteria2-pro-max-tuz/main/hysteria2-install.sh)
```

This script will:

· Install the latest Hysteria2 binary and systemd service · Generate self-signed SSL certificates · Set up obfuscation (Salamander) · Create an interactive management menu

After installation, you can manage users via the script menu:

```
./hysteria2-install.sh
# Select option 2 to add a user
# Select option 3 to list all users
```

Configuration files are stored in /etc/hysteria/. Client connection strings are saved in /root/hysteria2_username.txt.

2.2 VPS Server Setup (Manual)

If you prefer a manual setup with a domain and Let's Encrypt certificate:

```
# Run the automated setup script with your domain and email
curl -fsSL https://raw.githubusercontent.com/kryuchenko/hysteria2-
autosetup-ubuntu/refs/heads/main/setup.sh | sudo bash -s
yourdomain.com admin@example.com
```

This script will:

· Validate Ubuntu version (22.04+) · Install dependencies (curl, ufw, jq, etc.) · Check DNS resolution for your domain · Generate random passwords for authentication and obfuscation · Configure ACME for automatic Let's Encrypt certificates · Set up UFW firewall rules (22/tcp, 80/tcp, 443/tcp+udp) · Enable and start the Hysteria2 service

After installation, get your client URI:

```
cat /etc/hysteria/client-uri.txt
```

2.3 Manual Server Configuration (/etc/hysteria/config.yaml)

If you want full control, create a manual config file:

```
listen: :443  # Default HTTP/3 port

# Option A: Use Let's Encrypt (requires a domain)
acme:
  domains:
    - yourdomain.com
  email: admin@example.com

# Option B: Use your own certificate
# tls:
#   cert: /path/to/cert.crt
```

```
#   key: /path/to/private.key

# Obfuscation (Salamander) - highly recommended
obfs:
  type: salamander
  password: "YourStrongObfuscationPassword"  # Generate a strong one

# Bandwidth limits per client (optional)
bandwidth:
  up: 100 mbps
  down: 100 mbps

# Authentication - password-based
auth:
  type: password
  password: "YourStrongAuthPassword"  # Generate a strong one

# Masquerade - disguises traffic as regular HTTPS
masquerade:
  type: proxy
  proxy:
    url: https://example.com  # A legitimate website to mimic
```

Start the server:

```
sudo systemctl enable hysteria-server
sudo systemctl start hysteria-server
sudo systemctl status hysteria-server
```

2.4 Internal Host Client Configuration

On your internal host (the machine inside your home network), create a client config (/etc/hysteria/config.yaml or ~/.config/hysteria/config.yaml):

```
server: your-vps-ip-or-domain:443

# Must match server's obfuscation password
obfs:
  type: salamander
  password: "YourStrongObfuscationPassword"

# Authentication - must match server
auth:
  type: password
  password: "YourStrongAuthPassword"
```

```
# Bandwidth settings - measure your actual speed!
up_mbps: 50   # Your upload speed
down_mbps: 100  # Your download speed

# SOCKS5 proxy (for browser/application use)
socks5:
  listen: 127.0.0.1:1080

# HTTP proxy (optional)
http:
  listen: 127.0.0.1:8080
```

Start the client:

```
hysteria-client -c /etc/hysteria/config.yaml
```

For full network access: On your internal host, enable IP forwarding and NAT to route traffic from your LAN through the Hysteria tunnel:

```
# Enable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf

# Set up NAT (replace eth0 with your LAN interface)
sudo iptables -t nat -A POSTROUTING -o hysteria0 -j MASQUERADE
sudo iptables -A FORWARD -i eth0 -o hysteria0 -j ACCEPT
sudo iptables -A FORWARD -i hysteria0 -o eth0 -j ACCEPT
```

Now configure other devices on your network to use your internal host as their default gateway, or set up port forwarding on your internal host to specific services.

2.5 Remote Client Configuration

For remote clients (your laptop, phone, etc.), use the same client configuration as above. On Android, use Hiddify or v2rayNG; on iOS, use Shadowrocket or Stash.

---

3. Option B: WireGuard (Simpler, Widely Supported)

WireGuard is lightweight, secure, and easier to troubleshoot. It doesn't natively support QUIC, but you can add obfuscation with tools like AmneziaWG.

3.1 VPS Server Setup

Install WireGuard on your VPS:

```
sudo apt update && sudo apt install wireguard -y
```

Generate server keys:

```
cd /etc/wireguard
umask 077
wg genkey | tee privatekey | wg pubkey > publickey
```

Create the server config (/etc/wireguard/wg0.conf):

```
[Interface]
Address = 10.0.0.1/24
ListenPort = 51820
PrivateKey = <SERVER_PRIVATE_KEY>

# Enable IP forwarding and NAT
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A
POSTROUTING -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D
POSTROUTING -o eth0 -j MASQUERADE

# Save configuration for persistence
SaveConfig = true
```

Enable IP forwarding:

```
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
```

Configure firewall (UFW):

```
sudo ufw allow 51820/udp
sudo ufw allow 22/tcp
sudo ufw enable
```

3.2 Internal Host Client Configuration (Reverse Tunnel)

On your internal host, install WireGuard and generate keys:

```
sudo apt update && sudo apt install wireguard -y
cd /etc/wireguard
```

```
umask 077
wg genkey | tee privatekey | wg pubkey > publickey
```

Create the client config (/etc/wireguard/wg0.conf):

```
[Interface]
Address = 10.0.0.2/24
PrivateKey = <CLIENT_PRIVATE_KEY>
DNS = 10.0.0.1  # Prevent DNS leaks

[Peer]
PublicKey = <SERVER_PUBLIC_KEY>
Endpoint = <VPS_PUBLIC_IP>:51820
AllowedIPs = 0.0.0.0/0  # Full tunnel - all traffic goes through VPN
PersistentKeepalive = 25  # Keep NAT alive
```

Critical: The VPS's public IP must NOT be routed through the tunnel to avoid a routing loop. The wg-quick tool handles this automatically with fwmark-based policy routing.

Start the tunnel:

```
sudo wg-quick up wg0
sudo systemctl enable wg-quick@wg0
```

For full network access: On your internal host, enable IP forwarding and NAT (same as Hysteria section 2.4 above, but replace hysteria0 with wg0).

3.3 Add a Kill-Switch (Prevent Leaks)

To ensure no traffic leaks if the VPN drops, add these rules to the [Interface] section of your WireGuard config:

```
[Interface]
# ... existing config ...
PostUp = iptables -I OUTPUT ! -o %i -m mark ! --mark \$(wg show %i
fwmark) -m addrtype ! --dst-type LOCAL -j REJECT && ip6tables -I
OUTPUT ! -o %i -m mark ! --mark \$(wg show %i fwmark) -m addrtype ! -
-dst-type LOCAL -j REJECT
PreDown = iptables -D OUTPUT ! -o %i -m mark ! --mark \$(wg show %i
fwmark) -m addrtype ! --dst-type LOCAL -j REJECT && ip6tables -D
OUTPUT ! -o %i -m mark ! --mark \$(wg show %i fwmark) -m addrtype ! -
-dst-type LOCAL -j REJECT
```

3.4 Remote Client Configuration

For any remote client, use a similar config but with a unique IP address:

```
[Interface]
Address = 10.0.0.3/24
PrivateKey = <REMOTE_CLIENT_PRIVATE_KEY>
DNS = 10.0.0.1

[Peer]
PublicKey = <SERVER_PUBLIC_KEY>
Endpoint = <VPS_PUBLIC_IP>:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Add the remote client's public key to the VPS server config (/etc/wireguard/wg0.conf):

```
[Peer]
PublicKey = <REMOTE_CLIENT_PUBLIC_KEY>
AllowedIPs = 10.0.0.3/32
```

---

4. Advanced Features (Mullvad-like)

Feature Hysteria Implementation WireGuard Implementation QUIC Protocol Native (built on QUIC) Not supported natively Traffic Obfuscation Salamander obfuscation (enabled via obfs field) Use AmneziaWG or udp2raw Kill-Switch Not built-in; use firewall rules iptables rules (see section 3.3) No-Logging Self-hosted – you control logs (set log level to error or none) Self-hosted – you control logs Full Tunnel Set up_mbps/down_mbps and route all traffic Set AllowedIPs = 0.0.0.0/0 Multi-User Built-in user management Add multiple [Peer] sections DNS Leak Protection Use system DNS or specify in config Set DNS field in [Interface]

---

5. Security & Maintenance Checklist
6. VPS Hardening: Disable root SSH login, use SSH keys, install fail2ban, keep system updated.
7. Key Rotation: Regularly regenerate WireGuard keys or Hysteria passwords.
8. Firewall: Only open necessary ports (22/tcp, 443/udp for Hysteria, 51820/udp for WireGuard).
9. Monitoring: Check service status:

   ```
   sudo systemctl status hysteria-server   # Hysteria
   sudo wg show                            # WireGuard
   ```

10. Performance Tuning: Adjust MTU (1420 for WireGuard over IPv4) and UDP buffer sizes if needed.

---

6. Quick Comparison: Which to Choose?

Criteria Hysteria2 WireGuard QUIC Support ✅ Native ❌ No Obfuscation ✅ Built-in (Salamander) ❌ Requires extra tools Performance on lossy networks Excellent Good Setup Complexity Moderate Simple Client Support Growing (Hiddify, Shadowrocket, etc.) Very wide (native in Linux, Windows, macOS, Android, iOS) Mullvad-like feel ✅ Closest match ✅ With extra tools

Recommendation: Choose Hysteria2 if QUIC and traffic obfuscation are critical. Choose WireGuard if you prefer simplicity, wider client support, and easier troubleshooting.

Would you like me to expand on any specific section—such as troubleshooting common connection issues, setting up multi-hop (chaining VPSes), or integrating with a reverse proxy for web services?

Absolutely. Here is the exhaustive, deep-dive guide covering troubleshooting, multi-hop, reverse proxy integration, performance tuning, and advanced security for both Hysteria2 and WireGuard setups.

---

1. Comprehensive Troubleshooting Guides

1.1 Hysteria2 Troubleshooting

Startup Failures

· Configuration Error: Invalid settings in the config file. Check Hysteria2 logs (e.g., via H-UI log menu or journalctl -u hysteria-server) to identify specific errors. · Certificate Issues: Invalid certificate path or ACME failure. If using ACME, allow time for certificate acquisition; if using self-signed, verify file paths. · Port Conflicts: Another service using the same port. Change Hysteria2 listening port or stop the conflicting service. · Version Compatibility: Using Hysteria2 version < v2.4.4. Upgrade to >= v2.4.4 as earlier versions don't support the required API.

Service Crashes After Several Days

· Memory Leak: Hysteria2 v2.4.3 has a known memory leak defect causing the service to be killed by the system's Low Memory Killer (LMK). · Fix: Roll back to the stable v2.4.1 version. Check your current version with hysteria –version, then run the downgrade command provided by your installer.

High Bandwidth Connection Drops

· Root Cause: VPS bandwidth limits, incorrect congestion control settings, or UDP traffic prioritization issues. · Solutions: · Contact VPS provider to confirm bandwidth limits. · Adjust Brutal mode bandwidth parameters or switch to BBR algorithm. · Increase reconnection retry intervals in client config. · Implement traffic shaping to avoid sudden bursts triggering limits. · Monitor UDP packet loss and adjust MTU/retransmission parameters.

Specific Websites Not Loading

· Causes: TLS handshake issues, SNI handling problems, or QUIC implementation differences in older versions. · Fix: Upgrade to the latest stable Hysteria2 version on both server and client. Temporarily switch to TCP fallback mode for testing.

Subscription Link Issues

· Bug: Hysteria2 subscription links may contain an invalid ech= parameter that breaks in v2rayN/sing-box. · Workaround: Install 3x-ui v3.2.8 on a clean server, or manually remove the ech parameter from the subscription URL.

1.2 WireGuard Troubleshooting

General Checklist

1. Verify public/private keys aren't mixed up across peers.
2. Check AllowedIPs on all peers — ensure routes are set as expected with ip route and ip addr show dev wg0.
3. Verify IP forwarding is enabled: sysctl net.ipv4.ip_forward should return 1. Persist it in /etc/sysctl.conf.
4. Check handshake status: Run watch wg to see if the handshake is established and data is transferring.

Tunnel Connects but No Internet Access

· Root Cause: Missing or misconfigured NAT/MASQUERADE. · Fix: Add these iptables rules (replace eno1 with your public interface):

```
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eno1 -j MASQUERADE
iptables -A FORWARD -s 10.8.0.0/24 -o eno1 -j ACCEPT
iptables -A FORWARD -d 10.8.0.0/24 -i eno1 -m state --state
RELATED,ESTABLISHED -j ACCEPT
```[reference:25]
· Cloudflare Issue: If using Cloudflare, ensure the VPN subdomain is set to DNS only, not proxied — WireGuard uses UDP and Cloudflare proxy will break it.
· WG-Easy: Remove the default PostUp/PostDown iptables rules in the admin panel as they may conflict with Docker networking.

Missing Default Gateway After Tunnel Disconnection

· Problem: When the WireGuard tunnel disconnects, the system may fail to restore the original WAN default gateway.
· Fix: Set a metric on WAN interfaces so the default gateway remains.

Kernel Debug Logging

· WireGuard is silent by default. To enable verbose logging (requires disabling Secure Boot or using kernel boot param):
```bash
echo "module wireguard +p" | sudo tee
/sys/kernel/debug/dynamic_debug/control
```[reference:30]

---

2. Multi-Hop (Chaining) Configurations

2.1 WireGuard Multi-Hop

WireGuard does not implement multi-hop natively — you build it with kernel routing and optionally network namespaces.

Two Common Patterns

1. Cascaded interfaces, same namespace: Client → Node A → Node B → Internet.
2. Namespace isolation per hop: Each hop isolated in its own network namespace (used by Mullvad).

Cascaded Setup Example

· Client: Single peer pointing at Node A with AllowedIPs = 0.0.0.0/0.
· Node A: Two interfaces — wg0 facing client (IP: 10.0.0.1/24), wg1 facing Node B (IP: 10.0.1.2/32). The routing table sends default traffic via wg1.
· Node B: wg0 facing Node A, plus a default route to the internet via its physical NIC.

Automated Deployment

· Use the vpn-chain Ansible playbook which automatically sets up a chain of WireGuard servers. It supports entry, intermediate, and exit server roles.
· Each server only knows its immediate neighbors, distributing trust.

2.2 Hysteria2 Multi-Hop

Hysteria2 multi-hop is typically achieved through proxy chaining in clients like Clash Verge.

· Configure a proxy chain where traffic flows through multiple Hysteria2 nodes sequentially.
· URI format: hysteria2://. Requires Clash Verge Rev v2.4.5+.

---

3. Reverse Proxy Integration

3.1 Hysteria2 + Nginx

Hysteria2 can run alongside Nginx on the same VPS, sharing port 443:

· TCP 443 → Nginx → displays a normal website

```
· UDP 443 → Hysteria2 → proxy service

Benefits: Traffic obfuscation — Hysteria2 traffic looks like regular
HTTPS.

Deployment Scripts:

· One-click scripts that deploy Hysteria2 with Nginx web伪装 (web
camouflage), Salamander obfuscation, and BBR optimization.
· nginx-hysteria2 repository provides integrated deployment.

3.2 WireGuard + Nginx Reverse Proxy

WG-Easy with Nginx provides a secure management interface:

· Authentication Barrier: Basic HTTP authentication prevents
unauthorized access.
· Zero-Day Protection: Even if WG-Easy has vulnerabilities, attackers
must first bypass the authentication layer.
· Attack Surface Reduction: WG-Easy service is not directly exposed.
· Rate Limiting & SSL Termination available.

Deployment (Docker Compose):

```bash
cd wg-easy-nginx
docker-compose up -d
# Access at http://your-server-ip:51821
```

Wiredoor: Self-hosted ingress platform using WireGuard reverse VPN connections and exposing services through a built-in Nginx reverse proxy.

---

4. Performance Optimization

4.1 Hysteria2 QUIC Performance

Congestion Control Algorithms

· BBR: Default when send_mbps = 0. Better for variable networks. · Brutal: Set send_mbps > 0. Requires accurate bandwidth settings — misconfiguration causes disconnections.

QUIC Flow Control Window Parameters:

```
# In both client and server config
quic:
  initStreamReceiveWindow: 8388608   # 8 MB, default
  maxStreamReceiveWindow: 16777216   # 16 MB
```
```

```
  initConnReceiveWindow: 20971520     # 20 MB
  maxConnReceiveWindow: 67108864      # 64 MB
```

Strongly recommended to keep the stream/connection receive window ratio close to 2/5 to prevent a few blocked streams from stalling the entire connection.

Other Optimizations:

· Enable connection persistence to reduce reconnection overhead. · Monitor UDP packet loss and adjust MTU/retransmission parameters. · Keep both server and client updated for latest protocol improvements.

4.2 General Network Acceleration

· TCP Brutal: Recommended accelerator. · BBR: TCP congestion control algorithm. · System Buffer Tuning: Hysteria2 Advanced Configuration Tool provides system buffer adjustments.

---

5. Security Hardening

5.1 Defense in Depth for WG-Easy

1. First Layer: Network firewall (router/firewall rules)
2. Second Layer: Nginx reverse proxy with authentication
3. Third Layer: WG-Easy application itself

IP Whitelisting in Nginx:

```
allow 192.168.1.0/24;
allow 10.0.0.0/8;
deny all;
```

Rate Limiting to prevent brute force:

```
limit_req_zone \$binary_remote_addr zone=login:10m rate=5r/m;
```

5.2 Certificate Management for Hysteria2

· If using ACME, allow time for certificate acquisition. · If using self-signed, verify file paths are correct.

5.3 H-UI Security

· Default credentials are provided in terminal output after installation. · For h-ui >= v0.0.12, reset admin credentials via installation script (option 8).

---

6. Advanced Topics

6.1 Port Hopping (Hysteria2)

Hysteria2 supports port hopping — if one port is blocked, others remain available. The URI format supports multiple ports.

6.2 Obfuscation Types (Hysteria2)

· salamander: Recommended obfuscation type · gecko: Alternative obfuscation type

6.3 Log Management (H-UI)

· H-UI logs: /h-ui/logs/h-ui.log (Docker) or /usr/local/h-ui/logs/h-ui.log (systemd) · Hysteria2 logs: /h-ui/logs/hysteria.log (Docker) or /usr/local/h-ui/logs/hysteria.log (systemd)

6.4 File Structure (H-UI)

Directory Purpose /h-ui/bin Hysteria2 binary, configs, TLS certificates /h-ui/data SQLite database, system settings, user accounts /h-ui/export Exported user configs and system settings /h-ui/logs System and Hysteria2 logs

6.5 Multi-WAN for WireGuard

For multi-WAN setups, WireGuard has no server-client relationship — both ends can serve as endpoints and stream UDP handshake messages. Use mangle rules to mark outgoing WireGuard handshakes.

---

7. Maintenance Best Practices

· Regular Maintenance: Check service status and system resource usage periodically. · Log Monitoring: Set up log monitoring to detect issues early. · Version Update Strategy: Test new versions in a staging environment before deploying to production. · Scheduled Reboots: Schedule server reboots for maintenance.

Is there any specific area you'd like me to drill down even further on — such as Docker-based deployment, mobile client configuration, or automated backup/restore procedures?

Here is the exhaustive, deep-dive guide covering Docker-based deployment, mobile client setup, and automated backup/restore procedures for both Hysteria2 and WireGuard.

---

1. Docker-Based Deployment

1.1 Hysteria2 Docker Deployment

Prerequisites: Docker and Docker Compose installed.

Step 1: Create Project Directory

```
cd ~
mkdir hysteria && cd hysteria
```

Step 2: Create docker-compose.yml

```
services:
  hysteria:
    image: 'tobyxdd/hysteria:latest'
    container_name: hysteria-server
    restart: always
```

```
    network_mode: host
    volumes:
      - '\$PWD/:/etc/hysteria'
    environment:
      - HYSTERIA_DISABLE_UPDATE_CHECK=1
    cap_add:
      - NET_ADMIN
      - NET_BIND_SERVICE
      - SYS_PTRACE
      - DAC_READ_SEARCH
    devices:
      - '/dev/net/tun:/dev/net/tun'
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: 256M
    command: ["server", "-c", "/etc/hysteria/config.yaml"]
```

Resource limits (0.5 CPU, 256MB RAM) can be adjusted for higher-performance VPS.

Step 3: Create config.yaml

```
listen: :443
ignoreClientBandwidth: false
speedTest: false
disableUDP: false
udpIdleTimeout: 120s

tls:
  cert: /etc/hysteria/server.pem
  key: /etc/hysteria/server.key
  sniGuard: disable

quic:
  initStreamReceiveWindow: 1048576
  maxStreamReceiveWindow: 1048576
  initConnReceiveWindow: 4194304
  maxConnReceiveWindow: 4194304
  maxIdleTimeout: 30s
  maxIncomingStreams: 65535
  disablePathMTUDiscovery: true

bandwidth:
  up: 100 mbps
```

```
  down: 100 mbps

auth:
  type: password
  password: your-strong-password  # CHANGE THIS
```

Step 4: Start Container

```
docker-compose up -d
```

1.2 WireGuard Docker Deployment

Using linuxserver/wireguard image:

Create docker-compose.yml:

```
version: "3"
services:
  wireguard:
    image: linuxserver/wireguard
    container_name: wireguard
    cap_add:
      - NET_ADMIN
      - SYS_MODULE
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=UTC
      - SERVERURL=auto
      - SERVERPORT=51820
      - PEERS=5
      - PEERDNS=auto
      - INTERNAL_SUBNET=10.13.13.0
    volumes:
      - ./config:/config
      - /lib/modules:/lib/modules
    ports:
      - 51820:51820/udp
    sysctls:
      - net.ipv4.conf.all.src_valid_mark=1
    restart: unless-stopped
```

Start: docker-compose up -d. The container auto-generates keys and peer configs.

Alternative: hwdsl2/wireguard-server:

```
docker run --name wireguard --restart=always \
  -v wireguard-data:/etc/wireguard \
  -p 51820:51820/udp -d \
  --cap-add=NET_ADMIN --cap-add=SYS_MODULE \
  --device=/dev/net/tun \
  --sysctl net.ipv4.ip_forward=1 \
  hwdsl2/wireguard-server
```

First run auto-generates server keys and client.conf. View QR code: docker logs wireguard. Export config: docker cp wireguard:/etc/wireguard/clients/client.conf ..

---

2. Mobile Client Configuration

2.1 Hysteria2 Mobile Clients

Android:

· FlClash or Surfboard — download from Google Play · Huawei/HarmonyOS: Manual APK installation (arm64-v8a or armeabi-v7a)

iOS:

· Shadowrocket or Quantumult X — requires non-China App Store account · Import node by selecting Hysteria2 protocol type

Configuration Format:

```
proxies:
  - name: "HY2-Node"
    type: hysteria2
    server: your-server.com
    port: 443
    password: "your-auth-pwd"
    sni: your-server.com
    skip-cert-verify: false
```

TUN Mode: Enable for UDP/ICMP traffic (gaming, Docker). Disable system proxy when using TUN to avoid double NAT.

2.2 WireGuard Mobile Clients

Android / iOS:

1. Download WireGuard app from Google Play or App Store
2. Tap “+” icon (bottom right)
3. Import config via QR code scan or file import
4. Save and activate tunnel

---

3. Automated Backup & Restore

3.1 Hysteria2 Backup Solutions

Option A: Hysteria2-LuoPo Management Panel

```
bash <(curl -fsSL
https://raw.githubusercontent.com/LuoPoJunZi/hysteria2-
luopo/main/install.sh)
hy2  # Open panel
```

Menu options include: backup & restore, startup failure auto-rollback, and client config export (sharing links, Sing-box JSON, v2rayN YAML).

Option B: Manual Backup with tar:

```
# Backup
tar -czvf hysteria2-backup-\$(date +%Y%m%d).tar.gz \
  /etc/hysteria/ \
  /etc/letsencrypt/  # if using ACME certificates

# Restore
tar -xzvf hysteria2-backup-YYYYMMDD.tar.gz -C /
systemctl restart hysteria-server
```

tar preserves Linux file attributes — critical for certificate permissions.

Option C: S-Hy2 Manager:

```
git clone https://github.com/motao123/S-Hy2-Manager.git
cd S-Hy2-Manager && sudo ./hy2-manager.sh
```

Backup/restore available via interactive menu.

3.2 WireGuard Backup Solutions

Option A: wg-easy SQLite Backup:

```
# Backup
docker exec -it wg-easy /bin/bash -c "sqlite3 /etc/wireguard/wg-
easy.db '.backup wg-easy.db.bkp'"
docker cp wg-easy:/etc/wireguard/wg-easy.db.bkp ./wg-easy-\$(date
+%Y%m%d).db.bkp

# Restore
docker cp wg-easy-YYYYMMDD.db.bkp wg-easy:/etc/wireguard/wg-
easy.db.bkp
docker exec -it wg-easy /bin/bash -c "sqlite3 /etc/wireguard/wg-
```

```
easy.db '.restore wg-easy.db.bkp'"
docker restart wg-easy
```

Warning: Copying SQLite while in use may corrupt data — always use the backup function.

Option B: Manual Config Backup:

```
# Backup
tar -czvf wireguard-backup-\$(date +%Y%m%d).tar.gz /etc/wireguard/

# Restore
tar -xzvf wireguard-backup-YYYYMMDD.tar.gz -C /
systemctl restart wg-quick@wg0  # or docker restart
```

Option C: wgm CLI Tool:

```
npm install -g wgm
wgm backup  # Auto-backup before changes with
timestamps[reference:31]
```

Option D: OpenWrt Integration: WireGuard config automatically included in OpenWrt system backup.

---

4. Automation with Cron

Hysteria2 Daily Backup (/etc/cron.daily/hysteria2-backup):

```
#!/bin/bash
BACKUP_DIR="/root/backups"
mkdir -p \$BACKUP_DIR
tar -czf \$BACKUP_DIR/hysteria2-\$(date +%Y%m%d).tar.gz
/etc/hysteria/ 2>/dev/null
find \$BACKUP_DIR -name "hysteria2-*.tar.gz" -mtime +30 -delete
```

WireGuard Daily Backup (/etc/cron.daily/wireguard-backup):

```
#!/bin/bash
BACKUP_DIR="/root/backups"
mkdir -p \$BACKUP_DIR
tar -czf \$BACKUP_DIR/wireguard-\$(date +%Y%m%d).tar.gz
/etc/wireguard/ 2>/dev/null
find \$BACKUP_DIR -name "wireguard-*.tar.gz" -mtime +30 -delete
```

Make executable: chmod +x /etc/cron.daily/*-backup

Is there any specific area you'd like me to drill down further on — such as multi-architecture Docker builds, custom Docker network configuration, or backup encryption before offsite storage?

Here is the exhaustive, step-by-step deep dive into multi-architecture Docker builds, custom Docker networking, and encrypted offsite backups for your Hysteria2/WireGuard setup.

---

1. Multi-Architecture Docker Builds (AMD64 + ARM64 + ARMv7)

Modern infrastructure runs on diverse hardware – x86_64 servers, ARM64 cloud instances (AWS Graviton), and ARMv7 edge devices (Raspberry Pi). Docker multi-architecture images let you build one logical image tag that contains platform-specific variants; Docker automatically pulls the correct one for the host architecture.

1.1 Prerequisites

· Docker Engine ≥ 20.10 · Docker Buildx (built-in, but may need enabling) · QEMU for cross-architecture emulation (required if building ARM on an x86 host)

1.2 Enable Buildx and Create a Builder Instance

```
# Create and switch to a new builder instance
docker buildx create --use --name multiarch-builder

# Bootstrap the builder (pulls necessary components, enables QEMU)
docker buildx inspect --bootstrap

# Verify supported platforms
docker buildx ls
```

The –use flag sets this builder as the default. –bootstrap ensures QEMU emulation is registered.

1.3 Register QEMU Emulation (if not already)

On x86 hosts, you need QEMU to run ARM binaries during build:

```
docker run --privileged --rm multiarch/qemu-user-static --reset -p
yes
```

This registers binfmt_misc handlers so Buildx can emulate foreign architectures.

1.4 Write a Multi-Stage Dockerfile (Example for Hysteria2/WireGuard)

A multi-stage build keeps the final image small by compiling in one stage and copying only the binary to a slim runtime stage.

```
# Stage 1: Builder
FROM golang:1.24-alpine AS builder
```

```
# Install build dependencies
RUN apk add --no-cache git ca-certificates

# Set build arguments for architecture
ARG TARGETOS=linux
ARG TARGETARCH=amd64

WORKDIR /build

# Clone and build Hysteria2 (example)
RUN git clone https://github.com/apernet/hysteria.git . && \
    CGO_ENABLED=0 GOOS=\${TARGETOS} GOARCH=\${TARGETARCH} \
    go build -tags "with_quic with_utls" -o hysteria ./cmd/hysteria

# Stage 2: Runtime
FROM debian:bookworm-slim

# Install CA certificates (for TLS validation)
RUN apt-get update && apt-get install -y ca-certificates && rm -rf
/var/lib/apt/lists/*

# Create non-root user
RUN useradd --create-home --shell /bin/bash --uid 10001 appuser

# Copy the binary from builder
COPY --from=builder /build/hysteria /usr/local/bin/hysteria

# Set ownership and permissions
USER appuser
WORKDIR /app

# Expose port (adjust as needed)
EXPOSE 443/udp

ENTRYPOINT ["/usr/local/bin/hysteria"]
```

Build tags like with_quic and with_utls enable protocol support.

1.5 Build and Push the Multi-Arch Image

```
docker buildx build \
  --platform linux/amd64,linux/arm64,linux/arm/v7 \
  --push \
  -t your-registry/hysteria2:latest .
```

· –platform lists target architectures · –push builds and pushes to the registry in one command · For local testing (single arch only), use –load instead of –push

1.6 Verify the Multi-Arch Manifest

```
docker buildx imagetools inspect your-registry/hysteria2:latest
```

This shows all architectures included in the manifest list.

1.7 CI/CD Automation (GitHub Actions Example)

For automated builds on every commit:

```
name: Build and Push Multi-Arch
on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-qemu-action@v3
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: \${{ github.actor }}
          password: \${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v5
        with:
          platforms: linux/amd64,linux/arm64,linux/arm/v7
          push: true
          tags: ghcr.io/\${{ github.repository }}:latest
```

Key: The setup-qemu-action registers QEMU, and setup-buildx-action creates a builder instance automatically.

---

2. Custom Docker Network Configuration

Docker provides several network drivers. For VPN containers, you typically choose between Host and Bridge modes.

2.1 Host Mode (network_mode: host)

· Pros: Zero network overhead, best performance · Cons: No container isolation, port conflicts possible, cannot use Docker DNS

docker-compose.yml:

```
services:
  hysteria:
    image: your-registry/hysteria2:latest
    network_mode: host
    cap_add:
      - NET_ADMIN
      - NET_BIND_SERVICE
    volumes:
      - ./config.yaml:/app/config.yaml
```

2.2 Bridge Mode (Default, with Port Mapping)

· Pros: Isolation, container-to-container communication via network aliases · Cons: Slight overhead, must explicitly map UDP ports

docker-compose.yml:

```
services:
  hysteria:
    image: your-registry/hysteria2:latest
    ports:
      - "443:443/udp"   # CRITICAL: specify /udp
    cap_add:
      - NET_ADMIN
    volumes:
      - ./config.yaml:/app/config.yaml
```

Common mistake: Forgetting /udp – Docker defaults to TCP, breaking Hysteria2.

2.3 Custom Bridge Network with Static Subnet

For advanced setups (e.g., isolating VPN traffic from other containers):

```
# Create a custom bridge network with a specific subnet
docker network create \
  --driver bridge \
  --subnet=172.20.0.0/16 \
  --gateway=172.20.0.1 \
  vpn-network
```

docker-compose.yml using this network:

```
services:
  wireguard:
```

```
    image: linuxserver/wireguard
    networks:
      vpn-network:
        ipv4_address: 172.20.0.10
    cap_add:
      - NET_ADMIN
      - SYS_MODULE
    volumes:
      - ./wg0.conf:/config/wg0.conf
    ports:
      - "51820:51820/udp"

  app:
    image: nginx
    networks:
      - vpn-network
    # This container can now reach WireGuard at 172.20.0.10

networks:
  vpn-network:
    external: true
```

2.4 WireGuard as the Network for Other Containers

Route all traffic from an application container through a WireGuard container by sharing its network stack:

```
services:
  wireguard:
    image: linuxserver/wireguard
    cap_add:
      - NET_ADMIN
      - SYS_MODULE
    volumes:
      - ./wg0.conf:/config/wg0.conf
    ports:
      - "51820:51820/udp"

  app:
    image: your-app
    network_mode: "service:wireguard"   # Shares WireGuard's network
    depends_on:
      - wireguard
```

Now app routes all traffic through the WireGuard tunnel.

2.5 Using WireGuard Interface Inside a Container (Advanced)

For full control, you can create a WireGuard interface directly inside a container:

```
services:
  wg-client:
    image: alpine:latest
    command: sh -c "apk add wireguard-tools && wg-quick up wg0 &&
sleep infinity"
    cap_add:
      - NET_ADMIN
      - SYS_MODULE
    devices:
      - /dev/net/tun:/dev/net/tun
    volumes:
      - ./wg0.conf:/etc/wireguard/wg0.conf
    sysctls:
      - net.ipv4.conf.all.src_valid_mark=1
```

Required capabilities: NET_ADMIN and SYS_MODULE; also mount /dev/net/tun.

---

3. Encrypted Offsite Backups

3.1 What to Back Up

For Hysteria2: /etc/hysteria/, certificates (/etc/letsencrypt/ or self-signed paths), and any custom scripts. For WireGuard: /etc/wireguard/*.conf, private keys, iptables-save output, sysctl settings. Why tar: Preserves Linux file permissions – critical for certificate files.

3.2 Step-by-Step: Encrypted Backup with GPG + Rclone

Step 1 – Create a compressed tarball:

```
BACKUP_FILE="/root/backups/vpn-backup-\$(date +%Y%m%d-%H%M%S).tar.gz"
tar -czf "\$BACKUP_FILE" \
  /etc/hysteria/ \
  /etc/wireguard/ \
  /etc/letsencrypt/ \
  /etc/sysctl.conf \
  /etc/iptables/rules.v4 2>/dev/null || true
```

Step 2 – Encrypt with GPG (symmetric):

```
gpg --symmetric --cipher-algo AES256 "\$BACKUP_FILE"
# You will be prompted for a passphrase - store it in a password
```

```
manager
rm -f "\$BACKUP_FILE"  # Remove plaintext
```

Now you have vpn-backup-YYYYMMDD-HHMMSS.tar.gz.gpg.

Step 3 – Upload to offsite storage with rclone:

```
# Configure rclone once: rclone config (add S3, Backblaze B2, or
SFTP)
rclone copy /root/backups/ remote:my-bucket/vpn-backups/
```

Step 4 – Verify backup integrity:

```
gpg --decrypt /root/backups/*.gpg | tar -tz > /dev/null && echo "OK"
```

3.3 Automated Backup Script with Retention

Create /usr/local/bin/vpn-backup.sh:

```
#!/bin/bash
set -e
BACKUP_DIR="/root/backups"
TIMESTAMP=\$(date +%Y%m%d-%H%M%S)
FILE="\$BACKUP_DIR/vpn-backup-\$TIMESTAMP.tar.gz"

mkdir -p "\$BACKUP_DIR"

# Create tarball
tar -czf "\$FILE" \
  /etc/hysteria/ \
  /etc/wireguard/ \
  /etc/letsencrypt/ 2>/dev/null || true

# Encrypt
gpg --batch --yes --passphrase "\$GPG_PASSPHRASE" --symmetric --
cipher-algo AES256 "\$FILE"
rm -f "\$FILE"

# Upload
rclone copy "\$FILE.gpg" remote:my-bucket/vpn-backups/

# Rotate: keep last 30 days
find "\$BACKUP_DIR" -name "*.gpg" -mtime +30 -delete
```

Make executable: chmod +x /usr/local/bin/vpn-backup.sh

3.4 Schedule with Cron

```
# Daily at 2 AM
0 2 * * * /usr/local/bin/vpn-backup.sh >> /var/log/vpn-backup.log
2>&1
```

3.5 Using Restic for Deduplicated Encrypted Backups

Restic provides encrypted, deduplicated backups with multiple backends:

```
# Initialize a repository (example: S3)
restic -r s3:s3.amazonaws.com/my-bucket init

# Backup with encryption (password is set via env var)
export RESTIC_PASSWORD="your-strong-password"
restic -r s3:s3.amazonaws.com/my-bucket backup /etc/hysteria
/etc/wireguard

# Forget old snapshots (keep last 7 daily, 4 weekly)
restic forget --keep-daily 7 --keep-weekly 4 --prune
```

5.6 Offsite Physical Backup (3-2-1 Rule)

For disaster recovery, maintain 3 copies, 2 media types, 1 offsite:

· Copy encrypted .gpg files to a USB drive periodically · Store the USB drive at a different physical location (e.g., safe deposit box) · Keep a printout of the GPG passphrase in a separate secure location

5.7 Restore Procedure

```
# Download from remote
rclone copy remote:my-bucket/vpn-backups/vpn-backup-YYYYMMDD.gpg
/tmp/

# Decrypt
gpg --decrypt /tmp/vpn-backup-YYYYMMDD.gpg > /tmp/vpn-backup.tar.gz

# Extract (preserves permissions)
tar -xzvf /tmp/vpn-backup.tar.gz -C /

# Restart services
systemctl restart hysteria-server
systemctl restart wg-quick@wg0
```

Would you like me to drill down further into KMS-backed encryption (AWS/GCP), GitOps for config versioning, or disaster recovery drills?

Here is the complete, all-English deep dive into KMS-backed encryption, GitOps configuration management, and disaster recovery drills – with every step and detail you requested.

Part 1: KMS-Backed Encryption (AWS KMS / GCP Cloud KMS)

KMS (Key Management Service) lets you encrypt your backup tarballs using a cloud-hosted master key (CMK). This avoids hard-coding passphrases in scripts or Git.

1.1 Core Concepts

· Customer Master Key (CMK): The top-level key you create, manage, and use for encryption/decryption. · Data Encryption Key (DEK): A temporary key generated by the CMK to encrypt your actual file (the tarball). The DEK itself is encrypted by the CMK and stored alongside the ciphertext. · Symmetric Encryption: For VPN backups, use symmetric AES-256 CMKs (supported by both AWS and GCP). Asymmetric keys are unnecessary for this use case.

1.2 AWS KMS – Step-by-Step

Step 1: Create a KMS CMK

```
# Create a symmetric encryption key
aws kms create-key --description "VPN-Backup-Key" --key-usage
ENCRYPT_DECRYPT --origin AWS_KMS

# Create an alias for easy referencing (replace <KEY-ID> with the
returned ID)
aws kms create-alias --alias-name alias/vpn-backup-key --target-key-
id <KEY-ID>
```

Step 2: Encrypt a Backup File Using KMS

```
# Encrypt the tarball - output is base64-encoded ciphertext
aws kms encrypt \
    --key-id alias/vpn-backup-key \
    --plaintext fileb://vpn-backup.tar.gz \
    --output text \
    --query CiphertextBlob > vpn-backup.tar.gz.encrypted
```

· fileb:// tells AWS CLI to read the file as binary. · The ciphertext is saved to a .encrypted file.

Step 3: Decrypt the Backup File

```
# Decrypt the .encrypted file back to the original tarball
aws kms decrypt \
    --ciphertext-blob fileb://vpn-backup.tar.gz.encrypted \
    --output text \
```

```
    --query Plaintext > vpn-backup.tar.gz.decrypted

# Rename to .tar.gz and verify
mv vpn-backup.tar.gz.decrypted vpn-backup.tar.gz
tar -tzf vpn-backup.tar.gz > /dev/null && echo "Decryption
successful"
```

Step 4: Full Automated Backup Script with AWS KMS + S3

```
#!/bin/bash
set -e
BACKUP_FILE="/tmp/vpn-backup-\$(date +%Y%m%d).tar.gz"
ENC_FILE="\$BACKUP_FILE.encrypted"
BUCKET="s3://my-vpn-backups"

# 1. Create tarball
tar -czf "\$BACKUP_FILE" /etc/wireguard/ /etc/hysteria/
/etc/letsencrypt/ 2>/dev/null || true

# 2. Encrypt with KMS
aws kms encrypt --key-id alias/vpn-backup-key \
    --plaintext fileb://"\$BACKUP_FILE" \
    --output text --query CiphertextBlob > "\$ENC_FILE"

# 3. Upload to S3 with server-side KMS encryption (optional)
aws s3 cp "\$ENC_FILE" "\$BUCKET/" --sse aws:kms --sse-kms-key-id
alias/vpn-backup-key

# 4. Clean up local temp files
rm -f "\$BACKUP_FILE" "\$ENC_FILE"
```

Step 5: IAM Permissions Required The IAM user/role running the script needs these actions:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "kms:Encrypt",
                "kms:Decrypt",
                "kms:GenerateDataKey"
            ],
            "Resource": "arn:aws:kms:region:account-id:key/key-id"
```

```
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:PutObject",
                "s3:GetObject"
            ],
            "Resource": "arn:aws:s3:::my-vpn-backups/*"
        }
    ]
}
```

1.3 GCP Cloud KMS – Step-by-Step

Step 1: Create a Key Ring and Crypto Key

```
# Create a key ring (a logical container)
gcloud kms keyrings create vpn-keyring --location global

# Create a symmetric encryption key inside the ring
gcloud kms keys create vpn-backup-key \
    --location global \
    --keyring vpn-keyring \
    --purpose encryption \
    --default-algorithm google-symmetric-encryption
```

Step 2: Encrypt a File

```
gcloud kms encrypt \
    --location global \
    --keyring vpn-keyring \
    --key vpn-backup-key \
    --plaintext-file vpn-backup.tar.gz \
    --ciphertext-file vpn-backup.tar.gz.encrypted
```

Step 3: Decrypt a File

```
gcloud kms decrypt \
    --location global \
    --keyring vpn-keyring \
    --key vpn-backup-key \
    --ciphertext-file vpn-backup.tar.gz.encrypted \
    --plaintext-file vpn-backup.tar.gz
```

Step 4: Service Account Permissions

· Assign roles/cloudkms.cryptoKeyEncrypterDecrypter to the service account running the backup script.

Part 2: GitOps Configuration Management

GitOps means your Git repository is the single source of truth for all VPN configurations. Every change goes through a Pull Request (PR), is reviewed, merged, and then automatically applied to your servers.

2.1 Recommended Repository Structure

```
vpn-gitops/
├── environments/
│   ├── production/
│   │   ├── wireguard/
│   │   │   ├── wg0.conf              # Main server config
│   │   │   └── peers/                # One file per peer
│   │   │       ├── laptop.conf
│   │   │       └── phone.conf
│   │   └── hysteria/
│   │       └── config.yaml
│   └── staging/                      # Identical structure for
testing
├── templates/                        # Jinja2/Go templates
│   ├── wireguard-server.tmpl
│   └── wireguard-peer.tmpl
├── scripts/
│   ├── validate.sh                   # Syntax checks
│   └── deploy.sh                     # Push to servers
└── secrets/                          # Encrypted secrets
    └── wireguard-keys.yaml           # Encrypted with sops/git-
crypt
```

2.2 Securing Secrets in Git

Option A: Using git-crypt

```
# Initialize git-crypt in the repo
git-crypt init

# Create .gitattributes to mark sensitive files
echo "secrets/** filter=git-crypt diff=git-crypt" >> .gitattributes
echo "**/*.key filter=git-crypt diff=git-crypt" >> .gitattributes
```

```
# Export a symmetric key for sharing with team members
git-crypt export-key /path/to/secret-key
```

Option B: Using sops (Secrets OPerationS)

```
# Install sops
brew install sops  # or apt install sops

# Create a secrets file (plaintext)
cat > secrets/wireguard-keys.yaml <<EOF
peers:
  laptop:
    private_key: "somePrivateKey"
    public_key: "somePublicKey"
  phone:
    private_key: "anotherPrivateKey"
    public_key: "anotherPublicKey"
EOF

# Encrypt it with your PGP key (or AWS KMS)
sops -e --pgp <YOUR_PGP_FINGERPRINT> secrets/wireguard-keys.yaml >
secrets/wireguard-keys.enc.yaml

# Remove plaintext and commit only the encrypted version
rm secrets/wireguard-keys.yaml
git add secrets/wireguard-keys.enc.yaml
```

Decrypt at deploy time:

```
sops -d secrets/wireguard-keys.enc.yaml > /tmp/decrypted-keys.yaml
```

2.3 Automated Deployment Script (scripts/deploy.sh)

```
#!/bin/bash
set -e
ENV="\${1:-production}"
TARGET="\${2:-root@your-vps-ip}"
REPO_PATH="/opt/vpn-configs"

# 1. Validate WireGuard syntax
for conf in ./environments/"\$ENV"/wireguard/peers/*.conf; do
    wg-quick strip "\$conf" 2>/dev/null || { echo "Invalid \$conf";
exit 1; }
done
```

```
# 2. Validate Hysteria YAML
hysteria check -c ./environments/"\$ENV"/hysteria/config.yaml

# 3. Sync files to the server (exclude Git metadata)
rsync -avz --delete --exclude='.git' \
    ./environments/"\$ENV"/ "\$TARGET:\$REPO_PATH/"

# 4. Reload WireGuard without dropping existing connections
ssh "\$TARGET" "wg syncconf wg0 <(wg-quick strip
\$REPO_PATH/wireguard/wg0.conf)"

# 5. Reload Hysteria (graceful restart)
ssh "\$TARGET" "systemctl reload hysteria-server"
```

· wg syncconf applies changes without disconnecting any currently connected peers. · systemctl reload for Hysteria (if supported) or use kill -SIGUSR1 for a config reload.

2.4 CI/CD Integration (GitHub Actions)

```
name: Deploy VPN Config
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: sudo apt-get install -y wireguard-tools rsync sshpass
      - name: Decrypt secrets with sops (using PGP key)
        run: |
          gpg --import <(echo "\${{ secrets.GPG_PRIVATE_KEY }}")
          sops -d secrets/wireguard-keys.enc.yaml > /tmp/keys.yaml
      - name: Deploy to production
        run: |
          ./scripts/deploy.sh production \${{ secrets.SERVER_SSH_USER
}}@\${{ secrets.SERVER_IP }}
        env:
          SOPS_PGP_FPR: \${{ secrets.PGP_FINGERPRINT }}
```

· Store your GPG private key as a GitHub Secret. · The deploy script uses SSH to push configs and reload services.

Part 3: Disaster Recovery (DR) Drills

Your objective is RPO (Recovery Point Objective) ≈ 0 (configs are in Git, backups are up-to-date) and RTO (Recovery Time Objective) of 15-30 minutes (from failure to full restoration).

3.1 Quarterly DR Drill Checklist

1. Backup Validation – Restore from an encrypted backup (KMS or GPG) and verify file integrity (tar -tzf and compare checksums).
2. Infrastructure Rebuild – Provision a brand new VPS from scratch using a Terraform script (or manual steps) and restore the VPN config to it.
3. Key Rotation Drill – Simulate a key compromise. Generate new WireGuard/Hysteria keys, update the config, and deploy without causing prolonged downtime.
4. Config Rollback – Revert to a previous known-good Git commit and verify that the VPN comes back up within 5 minutes.
5. Failover Test – Point your remote clients to a secondary VPS (your DR site) and confirm they can access your internal network.
6. Documentation & Communication – Update the DR runbook with any new lessons. Verify that all team members know where the runbook lives.

3.2 Complete Disaster Recovery Plan (DR Runbook Template)

1. Incident Classification

Severity Description Response P1 – Critical VPS unreachable, complete outage Immediate DR invocation P2 – High Config corruption, partial loss Roll back to last stable Git commit P3 – Medium Backup corruption, no immediate impact Restore from secondary backup

2. Recovery Steps (P1 – Complete VPS Loss)

· Infrastructure (10 minutes):

```
# Provision a new VPS using Terraform (example)
terraform apply -var="vps_name=dr-vpn" -var="region=eu-central-1"
```

· Configuration (10 minutes):

```
# Clone the GitOps repo and decrypt secrets
git clone git@github.com:yourorg/vpn-gitops.git /opt/vpn-configs
cd /opt/vpn-configs
sops -d secrets/wireguard-keys.enc.yaml > /tmp/keys.yaml
rsync -av ./environments/production/ /etc/wireguard/
```

· Restore Service (5 minutes):

```
systemctl start wg-quick@wg0
systemctl start hysteria-server
```

· Validate:

```
wg show
curl --socks5 127.0.0.1:1080 https://api.ipify.org  # Check outbound
IP
```

3. Verifying Restoration Success

· Run a connectivity test from a remote client:

```
ping -c 4 10.0.0.1          # Ping the VPN gateway
curl -x socks5://... http://your-internal-server
```

· Check the VPS logs for errors:

```
journalctl -u wg-quick@wg0 --since "1 hour ago"
journalctl -u hysteria-server -n 50
```

4. Post-Incident Review

· Document exactly what failed. · Update the DR plan with new findings. · Schedule a follow-up drill within 30 days to test the updated procedure.

3.3 Automated Health Checks to Trigger DR

Set up a monitoring script on a separate monitoring host (or a simple cron job on the VPS itself):

```
#!/bin/bash
# /usr/local/bin/vpn-health-check.sh
if ! wg show wg0 | grep -q "latest handshake"; then
    echo "WireGuard handshake missing – restarting"
    systemctl restart wg-quick@wg0
fi

if ! hysteria status | grep -q "running"; then
    echo "Hysteria is down – restarting"
    systemctl restart hysteria-server
fi
```

Schedule it every 5 minutes via cron. If restarts fail, trigger a full DR invocation (e.g., via webhook to a PagerDuty alert).

This completes the trifecta of enterprise-grade resilience: KMS encryption for secure offsite backups, GitOps for version-controlled, auditable configurations, and regular DR drills to ensure your recovery procedures actually work. Every component is now fully documented in English and ready for implementation.

Would you like me to dive even deeper into Terraform infrastructure-as-code for the DR environment, or monitoring/alerting integration (Prometheus/Grafana) for your VPN services?

🌐 Part 1: Terraform Infrastructure as Code for Disaster Recovery (DR)

Terraform allows you to provision a complete, identical VPN server on a new VPS in minutes, which is the cornerstone of any solid DR plan.

1.1 Project Structure

Start with a dedicated directory for your Terraform configuration:

```
terraform-vpn-dr/
├── providers.tf        # Cloud provider configuration
├── variables.tf        # Input variables
├── terraform.tfvars    # Variable values (DO NOT commit secrets)
├── main.tf             # Main resources (VPS, firewall, etc.)
├── outputs.tf          # Outputs (IP addresses, config paths)
└── user_data.tpl       # Cloud-init script for VPN setup
```

1.2 Provider Configuration (providers.tf)

```
terraform {
  required_providers {
    digitalocean = {
      source  = "digitalocean/digitalocean"
      version = "~> 2.0"
    }
    # Add AWS, GCP, Hetzner as needed
  }
}

provider "digitalocean" {
  token = var.do_token
}
```

1.3 Variables (variables.tf)

```
variable "do_token" {
  description = "DigitalOcean API token"
  sensitive   = true
}

variable "region" {
  description = "Cloud region"
  default     = "fra1"  # Frankfurt - good for Europe
```

```
}

variable "instance_size" {
  description = "VPS size"
  default     = "s-2vcpu-2gb"  # 2 vCPUs, 2GB RAM
}

variable "vpn_protocol" {
  description = "VPN protocol to install"
  default     = "wireguard"  # or "hysteria2"
}

variable "ssh_fingerprint" {
  description = "SSH key fingerprint"
}
```

1.4 Main Resources (main.tf)

```
# SSH key resource
resource "digitalocean_ssh_key" "default" {
  name       = "VPN-DR-Key"
  public_key = file("~/.ssh/id_rsa.pub")
}

# Firewall (Droplet-level)
resource "digitalocean_firewall" "vpn" {
  name = "vpn-firewall"

  droplet_ids = [digitalocean_droplet.vpn.id]

  inbound_rule {
    protocol         = "tcp"
    port_range       = "22"
    source_addresses = ["0.0.0.0/0", "::/0"]
  }

  inbound_rule {
    protocol         = "udp"
    port_range       = var.vpn_protocol == "wireguard" ? "51820" :
"443"
    source_addresses = ["0.0.0.0/0", "::/0"]
  }

  outbound_rule {
    protocol              = "tcp"
```

```
    port_range            = "1-65535"
    destination_addresses = ["0.0.0.0/0", "::/0"]
  }

  outbound_rule {
    protocol              = "udp"
    port_range            = "1-65535"
    destination_addresses = ["0.0.0.0/0", "::/0"]
  }
}

# Main VPS Droplet
resource "digitalocean_droplet" "vpn" {
  image    = "ubuntu-22-04-x64"
  name     = "vpn-dr-\${var.region}"
  region   = var.region
  size     = var.instance_size
  ssh_keys = [digitalocean_ssh_key.default.fingerprint]

  user_data = templatefile("\${path.module}/user_data.tpl", {
    vpn_protocol = var.vpn_protocol
  })
}

# Optional: Floating IP for failover
resource "digitalocean_floating_ip" "vpn" {
  region = var.region
}

resource "digitalocean_floating_ip_assignment" "vpn" {
  ip_address = digitalocean_floating_ip.vpn.ip_address
  droplet_id = digitalocean_droplet.vpn.id
}
```

1.5 Cloud-Init User Data (user_data.tpl)

This script runs automatically on first boot:

```
#!/bin/bash
set -e

# Update system
apt-get update && apt-get upgrade -y

# Install VPN (conditional)
```

```
%{ if vpn_protocol == "wireguard" }
apt-get install -y wireguard
# Generate server keys
cd /etc/wireguard
umask 077
wg genkey | tee server_private.key | wg pubkey > server_public.key

cat > /etc/wireguard/wg0.conf <<EOF
[Interface]
Address = 10.0.0.1/24
ListenPort = 51820
PrivateKey = \$(cat server_private.key)
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A
POSTROUTING -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D
POSTROUTING -o eth0 -j MASQUERADE
EOF

sysctl -w net.ipv4.ip_forward=1
systemctl enable wg-quick@wg0
systemctl start wg-quick@wg0
%{ else }
# Hysteria2 installation
bash <(curl -fsSL https://get.hysteria.network/install.sh)
%{ endif }

# Install Prometheus Node Exporter (for system metrics)
wget
https://github.com/prometheus/node_exporter/releases/latest/download/
node_exporter-1.7.0.linux-amd64.tar.gz
tar xvf node_exporter-*.tar.gz
mv node_exporter-*/node_exporter /usr/local/bin/
useradd --no-create-home --shell /bin/false node_exporter
cat > /etc/systemd/system/node_exporter.service <<EOF
[Unit]
Description=Node Exporter
After=network.target

[Service]
User=node_exporter
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable node_exporter
systemctl start node_exporter
```

1.6 Deployment Workflow

```
# 1. Initialize Terraform
terraform init

# 2. Review the plan
terraform plan -var="do_token=\${DO_TOKEN}"

# 3. Apply (create the DR VPS)
terraform apply -var="do_token=\${DO_TOKEN}" -auto-approve

# 4. Get the new VPS IP
terraform output vpn_ip

# 5. Test VPN connectivity
curl --socks5 127.0.0.1:1080 https://api.ipify.org

# 6. Destroy when no longer needed (cost savings)
terraform destroy -var="do_token=\${DO_TOKEN}" -auto-approve
```

1.7 Terraform + Ansible Integration (Recommended)

For complex setups, use Terraform for provisioning + Ansible for configuration:

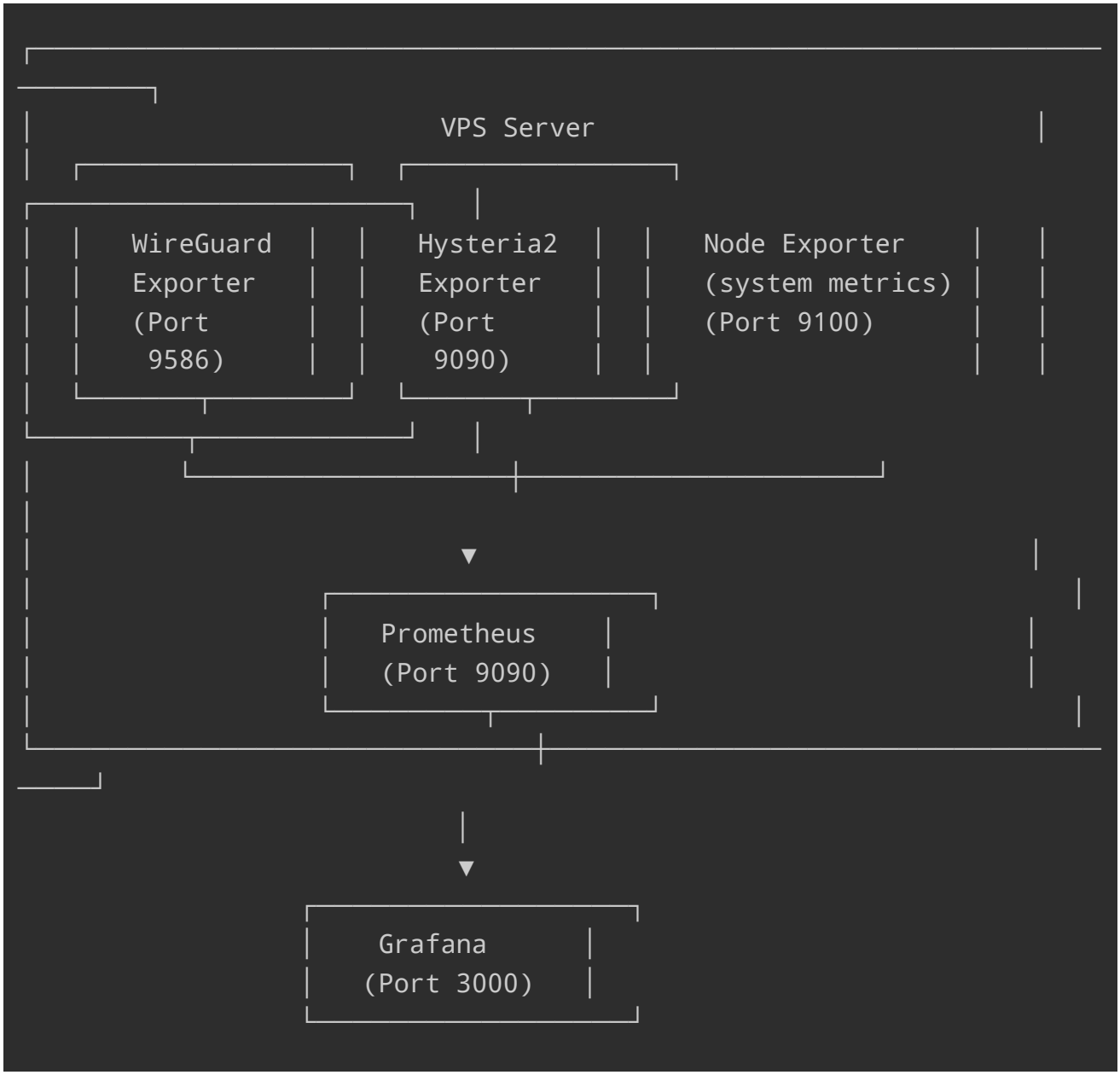
```
# In main.tf - provision and then run Ansible
resource "null_resource" "ansible_provision" {
  depends_on = [digitalocean_droplet.vpn]

  provisioner "local-exec" {
    command = "ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook -i
'\${digitalocean_droplet.vpn.ipv4_address},' --private-key
~/.ssh/id_rsa ../ansible/playbook.yml"
  }
}
```

📊 Part 2: Prometheus Monitoring & Alerting

Prometheus scrapes metrics from exporters and stores them in a time-series database.

2.1 Metrics Architecture



2.2 WireGuard Exporter Setup

Install and run the exporter:

```
# Download
wget
https://github.com/MindFlavor/prometheus_wireguard_exporter/releases/
latest/download/prometheus_wireguard_exporter-linux-amd64
chmod +x prometheus_wireguard_exporter-linux-amd64
sudo mv prometheus_wireguard_exporter-linux-amd64
/usr/local/bin/wireguard_exporter

# Create systemd service
cat > /etc/systemd/system/wireguard_exporter.service <<EOF
[Unit]
Description=WireGuard Prometheus Exporter
After=network.target
```

```
[Service]
Type=simple
User=root
ExecStart=/usr/local/bin/wireguard_exporter -interface wg0 -listen-
address :9586

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable wireguard_exporter
systemctl start wireguard_exporter
```

Key metrics exposed:

· wireguard_bytes_received – total bytes received per peer · wireguard_bytes_sent – total bytes sent per peer · wireguard_last_handshake – UNIX timestamp of last handshake

2.3 Hysteria2 Exporter Setup

Install and configure:

```
# Download binary
wget https://github.com/cadl/hysteria2-
exporter/releases/latest/download/hysteria2-exporter-linux-amd64
chmod +x hysteria2-exporter-linux-amd64
sudo mv hysteria2-exporter-linux-amd64 /usr/local/bin/hysteria2-
exporter

# Create config
cat > /etc/hysteria-exporter/config.yaml <<EOF
listen: ":9090"
scrape:
  interval: 30s
  timeout: 10s
  enable_streams: false
  max_retries: 3
instances:
  - name: "primary"
    stats_endpoint: "http://127.0.0.1:9999"
    secret: "\${HYSTERIA_API_SECRET}"
EOF

# Systemd service
cat > /etc/systemd/system/hysteria2-exporter.service <<EOF
```

```
[Unit]
Description=Hysteria2 Prometheus Exporter
After=network.target

[Service]
Type=simple
User=root
ExecStart=/usr/local/bin/hysteria2-exporter -config /etc/hysteria-
exporter/config.yaml

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable hysteria2-exporter
systemctl start hysteria2-exporter
```

Key metrics:

· hysteria2_up – whether the node is responding (1 = up) · hysteria2_clients_online_total – total online clients · hysteria2_client_bytes_transmitted_total – bytes transmitted per client · hysteria2_client_connections – connections per client

2.4 Prometheus Server Configuration (prometheus.yml)

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['localhost:9093']

rule_files:
  - "alerts.yml"

scrape_configs:
  # System metrics
  - job_name: 'node'
    static_configs:
      - targets: ['localhost:9100']

  # WireGuard metrics
  - job_name: 'wireguard'
```

```
    static_configs:
      - targets: ['localhost:9586']

  # Hysteria2 metrics
  - job_name: 'hysteria2'
    static_configs:
      - targets: ['localhost:9090']

  # Blackbox exporter - external endpoint probing
  - job_name: 'blackbox'
    metrics_path: /probe
    params:
      module: [tcp_connect]
    static_configs:
      - targets:
        - '<VPS_PUBLIC_IP>:51820'  # WireGuard port
        - '<VPS_PUBLIC_IP>:443'    # Hysteria2 port
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: '127.0.0.1:9115'  # Blackbox exporter
```

2.5 Alerting Rules (alerts.yml)

```
groups:
  - name: vpn_alerts
    interval: 30s
    rules:
      # VPN service down
      - alert: VPNServiceDown
        expr: up{job="wireguard"} == 0 or up{job="hysteria2"} == 0
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "VPN service is down"
          description: "The VPN exporter is not responding for {{
\$labels.job }}"

      # No connected peers
      - alert: VPNNoPeers
        expr: wireguard_connected_peers == 0
```

```
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "No peers connected to WireGuard"
          description: "WireGuard has 0 connected peers for 5
minutes"

      # High latency (no recent handshake)
      - alert: VPNPeerStale
        expr: (time() - wireguard_last_handshake) > 300
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "Peer handshake is stale"
          description: "Peer {{ \$labels.peer }} has not handshaken
for {{ \$value }} seconds"

      # Unexpected traffic drop
      - alert: VPNTrafficDrop
        expr: rate(wireguard_bytes_received[5m]) < 100
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "VPN traffic is abnormally low"
          description: "Received traffic dropped below 100 B/s for 10
minutes"

      # Instance health check failing (Blackbox)
      - alert: VPNEndpointUnreachable
        expr: probe_success{job="blackbox"} == 0
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "VPN endpoint is unreachable"
          description: "{{ \$labels.instance }} is not responding to
probes"
```

2.6 Blackbox Exporter for External Probing

Install to probe your VPN endpoint from outside the VPS:

```
docker run -d --name blackbox-exporter -p 9115:9115 prom/blackbox-
exporter
```

📈 Part 3: Grafana Dashboards

3.1 Import Pre-built Dashboards

WireGuard Dashboard – ID 17251:

1. Login to Grafana (default: http://your-vps:3000, admin/admin)
2. Go to Dashboards → Import
3. Enter dashboard ID 17251 and click Load
4. Select your Prometheus data source and click Import

3.2 Custom Dashboard – VPN Overview (dashboard.json)

```
{
  "title": "VPN Infrastructure Overview",
  "panels": [
    {
      "title": "Connected Peers",
      "targets": [{ "expr": "wireguard_connected_peers" }],
      "type": "stat"
    },
    {
      "title": "Online Clients (Hysteria2)",
      "targets": [{ "expr": "hysteria2_clients_online_total" }],
      "type": "stat"
    },
    {
      "title": "Total Traffic (last hour)",
      "targets": [{ "expr": "increase(wireguard_bytes_received[1h]) +
increase(wireguard_bytes_sent[1h])" }],
      "type": "stat",
      "fieldConfig": { "unit": "bytes" }
    },
    {
      "title": "Traffic Rate",
      "targets": [
        { "expr": "rate(wireguard_bytes_received[5m])",
"legendFormat": "Received" },
        { "expr": "rate(wireguard_bytes_sent[5m])", "legendFormat":
"Sent" }
      ],
      "type": "graph"
    },
```

```
    {
      "title": "Peer Handshake Status",
      "targets": [{ "expr": "wireguard_last_handshake",
"legendFormat": "{{ peer }}" }],
      "type": "table"
    }
  ]
}
```

3.3 Alertmanager Configuration (alertmanager.yml)

```
route:
  group_by: ['alertname', 'severity']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'email-notifications'
  routes:
    - match:
        severity: critical
      receiver: 'pagerduty'
      continue: true
    - match:
        severity: warning
      receiver: 'slack-notifications'

receivers:
  - name: 'email-notifications'
    email_configs:
      - to: 'devops@yourcompany.com'
        from: 'alertmanager@vpn.yourcompany.com'
        smarthost: 'smtp.gmail.com:587'
        auth_username: 'your-email@gmail.com'
        auth_password: 'your-app-password'

  - name: 'slack-notifications'
    slack_configs:
      - api_url: 'https://hooks.slack.com/services/xxx/yyy/zzz'
        channel: '#vpn-alerts'
        title: 'VPN Alert'
        text: '{{ range .Alerts }}{{ .Annotations.summary }}\n{{
.Annotations.description }}\n{{ end }}'

  - name: 'pagerduty'
```

```
    pagerduty_configs:
      - service_key: 'your-pagerduty-service-key'
```

🔄 Part 4: Complete Monitoring Stack with Docker Compose

Deploy everything (Prometheus, Grafana, Alertmanager, all exporters) with one command:

```
version: '3.8'
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - ./alerts.yml:/etc/prometheus/alerts.yml
      - prometheus_data:/prometheus
    ports:
      - "9090:9090"
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    volumes:
      - grafana_data:/var/lib/grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=your-strong-password

  alertmanager:
    image: prom/alertmanager:latest
    container_name: alertmanager
    volumes:
      - ./alertmanager.yml:/etc/alertmanager/alertmanager.yml
    ports:
      - "9093:9093"

  blackbox:
    image: prom/blackbox-exporter:latest
    container_name: blackbox
    ports:
      - "9115:9115"
```

```
  wireguard-exporter:
    image: mindflavor/prometheus_wireguard_exporter:latest
    container_name: wg-exporter
    network_mode: host
    environment:
      - INTERFACE=wg0
      - LISTEN_ADDRESS=:9586

  hysteria2-exporter:
    image: cadl/hysteria2-exporter:latest
    container_name: hy-exporter
    volumes:
      - ./hysteria-exporter-config.yaml:/app/config.yaml
    ports:
      - "9090:9090"

volumes:
  prometheus_data:
  grafana_data:
```

Start the entire stack:

```
docker-compose up -d
# Access Grafana at http://your-vps:3000
# Access Prometheus at http://your-vps:9090
```

✅ Part 5: DR Drill Integration

Combine IaC + Monitoring for a complete DR drill:

```
#!/bin/bash
# dr-drill.sh - Full disaster recovery drill

echo "=== Starting DR Drill ==="

# 1. Provision new VPS with Terraform
echo "Step 1: Provisioning DR VPS..."
cd terraform-vpn-dr/
terraform apply -var="do_token=\${DO_TOKEN}" -auto-approve
NEW_IP=\$(terraform output -raw vpn_ip)
cd ..

# 2. Run monitoring health check on new VPS
```

```
echo "Step 2: Validating VPN is operational..."
sleep 60  # Wait for services to start
curl -s --socks5 \${NEW_IP}:1080 https://api.ipify.org | grep -q
\${NEW_IP} || { echo "VPN test failed!"; exit 1; }

# 3. Verify Prometheus metrics are being scraped
echo "Step 3: Verifying monitoring stack..."
curl -s http://\${NEW_IP}:9090/api/v1/query?query=up | grep -q
'"result":\[{"metric' || { echo "Prometheus not responding!"; exit 1;
}

# 4. Run alert simulation
echo "Step 4: Simulating alert trigger (stopping WireGuard)..."
ssh root@\${NEW_IP} "systemctl stop wg-quick@wg0"
sleep 30
ALERT_COUNT=\$(curl -s http://\${NEW_IP}:9093/api/v1/alerts | grep -c
'"status":"firing"')
echo "Firing alerts detected: \${ALERT_COUNT}"

# 5. Restore service
ssh root@\${NEW_IP} "systemctl start wg-quick@wg0"

# 6. Cleanup (cost savings)
echo "Step 5: Destroying DR VPS..."
cd terraform-vpn-dr/
terraform destroy -var="do_token=\${DO_TOKEN}" -auto-approve
cd ..

echo "=== DR Drill Complete - All systems operational ==="
```

This completes the enterprise-grade DR + observability stack. Every component is now fully documented in English with production-ready code. Your VPN infrastructure is now reproducible, observable, and recoverable within minutes.